

filho 400
 filho direito 400
 filho esquerdo 400
 função dequeue de uma fila 395
 função enqueue de uma fila 395
 função free 381
 função malloc 381
 função predicado 388
 indireção dupla (ponteiro para um ponteiro) 388
 irmãos 400
 link (ponteiro em uma estrutura autorreferenciada) 381
 links 382
 listas encadeadas 380, 382
 na ordem 401
 nó folha 400
 nó pai 400
 nó raiz 400
 nó substituto 412
 nós 382
 notação infixa 409
 notação pós-fixa 409
 operador sizeof 381
 pilhas 380, 390
 ponteiro NULL 381
 ponteiro para ponteiro (indireção dupla) 388
 ponteiro para void (void *) 381
 pós-ordem 401
 pré-ordem 401
 primeiro a entrar, primeiro a sair (FIFO — first-in first-out) 395
 subárvore direita 400
 subárvore esquerda 400
 topo de uma pilha 380
 travessia de árvore binária por ordem de nível 413
 último a entrar, primeiro a sair (LIFO — last-in-first-out) 390

Exercícios de autorrevisão

12.1 Preencha os espaços em cada uma das sentenças:

- a) Uma estrutura _____ é usada para formar estruturas de dados dinâmicas.
 - b) A função _____ é usada para alocar memória dinamicamente.
 - c) Um(a) _____ é uma versão especializada de uma lista encadeada em que os nós podem ser inseridos e excluídos somente do início da lista.
 - d) Funções que se parecem com uma lista encadeada, mas não a modificam, são conhecidas como _____.
 - e) Uma fila é conhecida como uma estrutura de dados _____.
 - f) O ponteiro para o próximo nó em uma lista encadeada é conhecido como um(a) _____.
 - g) A função _____ é usada para reivindicar a memória alocada dinamicamente.
 - h) Um(a) _____ é uma versão especializada de uma lista encadeada em que os nós só podem ser inseridos no início da lista e excluídos do final da lista.
 - i) Um(a) _____ é uma estrutura de dados não linear, bidimensional, que contém nós com dois ou mais links.
 - j) Uma pilha é conhecida como uma estrutura de dados _____, pois o último nó inserido é o primeiro nó a ser removido.
 - k) Os nós de uma árvore _____ contêm dois membros de link.
 - l) O primeiro nó de uma árvore é o nó _____.
 - m) Cada link em um nó de árvore aponta para um(a) _____ ou um(a) _____ desse nó.
 - n) Um nó de árvore que não possui filhos é chamado de nó _____.
 - o) Os três algoritmos de travessia (explicados neste capítulo) para uma árvore binária são _____, _____ e _____.
- 12.2** Quais são as diferenças entre uma lista encadeada e uma pilha?
- 12.3** Quais são as diferenças entre uma pilha e uma fila?
- 12.4** Escreva uma instrução ou um conjunto de instruções para realizar cada uma das tarefas a seguir. Suponha que todas as manipulações ocorram em main (portanto, endereços de variáveis de ponteiro não são necessários), e considere as seguintes definições:
- ```

struct gradeNode {
 char lastName[20];
 double grade;
 struct gradeNode *nextPtr;
};

typedef struct gradeNode GradeNode;

typedef GradeNode *GradeNodePtr;

```
- a) Crie um ponteiro para o início da lista chamada startPtr. A lista está vazia.
  - b) Crie um novo nó do tipo GradeNode que seja apontado pelo ponteiro newPtr do tipo GradeNodePtr.

Atribua a string “Jones” ao membro `lastName` e o valor 91.5 ao membro `grade` (use `strcpy`). Inclua todas as declarações e instruções necessárias.

- c) Suponha que a lista apontada por `startPtr` atualmente consista em 2 nós — um contendo “Jones” e um contendo “Smith”. Os nós estão em ordem alfabética. Indique as instruções necessárias para inserir nós na ordem contendo os seguintes dados para `lastName` e `grade`:

|             |      |
|-------------|------|
| “Adams”     | 85.0 |
| “Thompson”  | 73.5 |
| “Pritchard” | 66.5 |

Use ponteiros `previousPtr`, `currentPtr` e `newPtr` para realizar as inserções. Informe para onde `previousPtr` e `currentPtr` apontam antes de cada inserção. Suponha que `newPtr` sempre aponte para o novo nó, e que o novo nó já tenha recebido dados.

- d) Escreva um loop `while` que imprima os dados em cada nó da lista. Use o ponteiro `currentPtr` para se movimentar pela lista.
- e) Escreva um loop `while` que exclua todos os nós na lista e libere a memória associada a cada nó. Use o ponteiro `currentPtr` e o ponteiro `tempPtr` para percorrer a lista e liberar a memória, respectivamente.

**12.5** Forneça as travessias na ordem, na pré-ordem e na pós-ordem da árvore de busca binária da Figura 12.22.

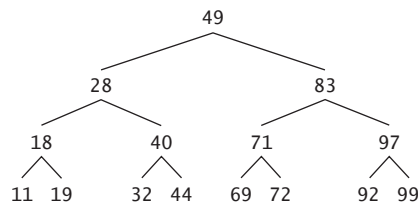


Figura 12.22 ■ Uma árvore de busca binária de 15 nós.

## ■ Respostas dos exercícios de autorrevisão

- 12.1** a) autorreferenciada. b) `malloc`. c) pilha. d) predicados. e) FIFO. f) link. g) `free`. h) fila. i) árvore. j) LIFO. k) binária. l) raiz. m) filho, subárvore. n) folha. o) in order, pré-ordem, pós-ordem.
- 12.2** É possível inserir um nó em qualquer lugar de uma lista encadeada e remover um nó de qualquer lugar em uma lista encadeada. Porém, os nós em uma pilha só podem ser inseridos no topo da pilha e removidos do tipo de uma pilha.
- 12.3** Uma fila tem ponteiros para sua cabeça e sua cauda, de modo que os nós podem ser inseridos na cauda e excluídos da cabeça. Uma pilha tem um único ponteiro para o topo da pilha, onde a inserção e a exclusão de nós são realizadas.
- 12.4** a) `GradeNodePtr startPtr = NULL;`  
 b) `GradeNodePtr newPtr;`  
`newPtr = malloc( sizeof( GradeNode ) );`  
`strcpy( newPtr->lastName, "Jones" );`  
`newPtr->grade = 91.5;`  
`newPtr->nextPtr = NULL;`
- c) Para inserir “Adams”:  
`previousPtr is NULL, currentPtr aponta para primeiro elemento na lista.`  
`newPtr->nextPtr = currentPtr;`  
`startPtr = newPtr;`  
 Para inserir “Thompson”:  
`previousPtr aponta para último elemento na lista (contendo “Smith”)`  
`currentPtr is NULL.`  
`newPtr->nextPtr = currentPtr;`  
`previousPtr->nextPtr = newPtr;`  
 Para inserir “Pritchard”:  
`previousPtr aponta para o nó contendo “Jones”`  
`currentPtr aponta para o nó contendo “Smith”`  
`newPtr->nextPtr = currentPtr;`  
`previousPtr->nextPtr = newPtr;`
- d) `currentPtr = startPtr;`  
`while ( currentPtr != NULL ) {`  
`printf( "Lastname = %s\nGrade =`

```

 %6.2f\n",
 currentPtr->lastName, currentPtr->grade);
 currentPtr = currentPtr->nextPtr;
}

```

```

e) currentPtr = startPtr;
while (currentPtr != NULL) {
 tempPtr = currentPtr;
 currentPtr = currentPtr->nextPtr;
 free(tempPtr);
}

```

```

 }
 startPtr = NULL;
}

```

## 12.5 A travessia in order é:

11 18 19 28 32 40 44 49 69 71 72 83 92 97 99

A travessia na pré-ordem é:

49 28 18 11 19 40 32 44 83 71 69 72 97 92 99

A travessia na pós-ordem é:

11 19 18 32 44 40 28 69 72 71 92 99 97 83 49

## Exercícios

- 12.6 Concatenação de listas.** Escreva um programa que concatene duas listas de caracteres encadeadas. O programa deverá incluir a função `concatenate`, que recebe ponteiros para as duas listas como argumentos e concatena a segunda lista com a primeira.
- 12.7 Mescla de listas ordenadas.** Escreva um programa que mescle duas listas ordenadas de inteiros em uma única lista ordenada de inteiros. A função `merge` deverá receber ponteiros para o primeiro nó de cada uma das listas a serem mescladas, e deverá retornar um ponteiro para o primeiro nó da lista mesclada.
- 12.8 Inserção em uma lista ordenada.** Escreva um programa que insira 25 inteiros aleatórios de 0 a 100 em ordem em uma lista encadeada. O programa deverá calcular a soma dos elementos e a média de ponto flutuante dos elementos.
- 12.9 Criação de uma lista encadeada seguida da inversão de seus elementos.** Escreva um programa que crie uma lista encadeada de 10 caracteres, e depois crie uma cópia da lista em ordem reversa.
- 12.10 Inversão de palavras de uma sentença.** Escreva um programa que aceite uma linha de texto e use uma pilha para imprimir a linha invertida.
- 12.11 Testador de palíndromos.** Escreva um programa que use uma pilha para determinar se uma string é um palíndromo (ou seja, se ela é escrita de forma idêntica de trás para a frente e de frente para trás). O programa deverá ignorar os espaços e os sinais de pontuação.
- 12.12 Conversão de infix para pós-fix.** As pilhas são usadas pelos compiladores para ajudar no processo de avaliação de expressões e na geração de código em linguagem de máquina. Neste e no próximo exercício, investigaremos como os compiladores avaliam as expressões aritméticas que consistem apenas em constantes, operadores e parênteses.

Os seres humanos geralmente escrevem expressões como  $3 + 4 \times 7 / 9$ , em que o operador (+ ou / aqui) é escrito entre seus operandos — isso é chamado de **notação infix**. Os computadores ‘preferem’ a **notação pós-fix**, na qual o operador é escrito à direita de seus dois operandos. As expressões infixas precedentes apareceriam em notação pós-fix como  $3 \ 4 \ + \ 7 \ 9 \ /$ , respectivamente.

Para calcular uma expressão infix complexa, um compilador teria de, primeiro, converter a expressão para a notação pós-fix e, então, analisar e calcular a versão pós-fix da expressão. Cada um desses algoritmos exige somente uma única passagem pela expressão, da esquerda para a direita. Cada algoritmo usa um objeto pilha para suportar sua operação e, em cada passagem do algoritmo, a pilha é usada com uma finalidade diferente.

Nesse exercício, você escreverá uma versão do algoritmo de conversão de notação infix para pós-fix. No próximo exercício, você escreverá uma versão do algoritmo de análise da expressão pós-fix.

Escreva um programa que converta uma expressão aritmética infix comum (suponha que uma expressão válida seja fornecida como entrada) com inteiros de um único dígito, tal como

$$(6 + 2) * 5 - 8 / 4$$

para uma expressão pós-fix. A versão pós-fix da expressão infix do exemplo anterior é

$$6 \ 2 \ + \ 5 \ * \ 8 \ 4 \ / \ -$$

O programa deverá ler a expressão para o array de caracteres `infix` e usar versões modificadas das funções de pilha implementadas neste capítulo para ajudar a criar a expressão pós-fix em um array de caracteres `postfix`. O algoritmo para se criar uma expressão pós-fix é o seguinte:

- 1) Insira um parêntese à esquerda ‘(’ no topo da pilha.
- 2) Acrescente um parêntese à direita ‘)’ ao final de `infix`.

- 3) Enquanto a pilha não estiver vazia, leia *infix* da esquerda para a direita, e faça o seguinte:

Se o caractere atual em *infix* for um dígito, copie-o para o próximo elemento de *postfix*.

Se o caractere atual em *infix* for um parêntese à esquerda, insira-o no topo da pilha.

Se o caractere atual em *infix* for um operador, Remova operadores (se existirem) do topo da pilha enquanto tiverem precedência igual ou mais alta que o operador atual, e insira os operadores removidos em *postfix*.

Insira o caractere atual em *infix* no topo da pilha.

Se o caractere atual em *infix* é um parêntese à direita

Remova operadores do topo da pilha e insira-os em *postfix* até um parêntese à esquerda estar no topo da pilha.

Remova (e descarte) da pilha o parêntese à esquerda.

As operações aritméticas a seguir são permitidas em uma expressão:

|   |               |
|---|---------------|
| + | adição        |
| − | subtração     |
| * | multiplicação |
| / | divisão       |
| ^ | exponenciação |
| % | módulo        |

A pilha deverá ser mantida por meio das seguintes declarações:

```
struct stackNode {
 char data;
 struct stackNode *nextPtr;
};
typedef struct stackNode StackNode;
typedef StackNode *StackNodePtr;
```

O programa deverá consistir em *main* e outras oito funções com os seguintes cabeçalhos:

```
void convertToPostfix(char infix[],
char postfix[])
```

Converte a expressão *infixa* para a notação pós-fixa.

```
int isOperator(char c)
```

Determina se *c* é um operador.

```
int precedence(char operator1, char
operator2)
```

Determina se a precedência de *operator1* é menor, igual ou maior que a precedência de *operator2*. A função retorna −1, 0 e 1, respectivamente.

```
void push(StackNodePtr *topPtr, char va-
lue)
```

Insere um valor na pilha.

```
char pop(StackNodePtr *topPtr)
```

Retira um valor da pilha.

```
char stackTop(StackNodePtr topPtr)
```

Retorna o valor do topo da pilha sem retirá-lo da pilha.

```
int isEmpty(StackNodePtr topPtr)
```

Determina se a pilha está vazia.

```
void printStack(StackNodePtr topPtr)
```

Imprime a pilha.

**12.13 Avaliador de notação pós-fixada.** Escreva um programa que avalie uma expressão pós-fixa (suponha que ela seja válida), tal como

6 2 + 5 \* 8 4 / −

O programa deve ler uma expressão pós-fixa que consista em dígitos isolados e operadores para um array de caracteres. Usando versões modificadas das funções de pilha implementadas anteriormente neste capítulo, o programa deve percorrer a expressão e calculá-la. O algoritmo é o seguinte:

- 1) Acrescente o caractere nulo ('\\0') ao fim da expressão pós-fixa. Quando o caractere nulo for encontrado, nenhum processamento adicional será necessário.
- 2) Enquanto '\\0' não for encontrado, leia a expressão, da esquerda para a direita.

Se o caractere atual for um dígito,

Insira seu valor inteiro no topo da pilha (o valor inteiro de um caractere que é um dígito é o seu valor no conjunto de caracteres do computador menos o valor de '0' no conjunto de caracteres do computador).

Caso contrário, se o caractere atual for um *operador*,

Remova os dois elementos do topo da pilha para as variáveis *x* e *y*.

Calcule *y operador x*.

Insira o resultado do cálculo no topo da pilha.

- 3) Quando o caractere nulo for encontrado na expressão, remova o valor do topo da pilha. Este é o resultado da expressão pós-fixa.

[Nota: na etapa (2) anterior, se o operador for '/', o topo da pilha será 2, e o próximo elemento na pilha será 8, então remova 2 da pilha e atribua a *x*, remova 8 para *y*, calcule 8/2 e insira o resultado, 4, de volta no topo da pilha. Essa nota também se aplica ao operador '−'.]

As operações aritméticas permitidas em uma expressão são:

|   |               |
|---|---------------|
| + | adição        |
| − | subtração     |
| * | multiplicação |

/      divisão  
 ^      exponenciação  
 %      módulo

A pilha deve ser mantida com as seguintes declarações:

```
struct stackNode {
 int data;
 struct stackNode *nextPtr;
};
typedef struct stackNode StackNode;
typedef StackNode *StackNodePtr;
```

O programa deve consistir em main e em outras seis funções com os seguintes cabeçalhos:

```
int evaluatePostfixExpression(char
*expr)
 Avalia a expressão pós-fixa.

int calculate(int op1, int op2, char
operator)
 Avalia a expressão op1 operador op2.

void push(StackNodePtr *topPtr, int
value)
 Insere um valor na pilha.

int pop(StackNodePtr *topPtr)
 Retira um valor da pilha.

int isEmpty(StackNodePtr topPtr)
 Determina se a pilha está vazia.

void printStack(StackNodePtr topPtr)
 Imprime a pilha.
```

**12.14 Modificação do avaliador de notação pós-fixada.** Modifique o programa avaliador de pós-fixado do Exercício 12.13 para que possa processar operandos inteiros maiores que 9.

**12.15 Simulação de supermercado.** Escreva um programa que simule uma fila no caixa de um supermercado. A fila é um objeto fila. Os clientes chegam em intervalos inteiros aleatórios de 1 a 4 minutos. Além disso, cada cliente é atendido em intervalos inteiros aleatórios de 1 a 4 minutos. Obviamente, as taxas precisam ser equilibradas. Se a taxa de chegada média é maior que a taxa de atendimento médio, a fila crescerá infinitamente. Mesmo com taxas 'equilibradas', a aleatoriedade ainda pode causar longas filas. Execute a simulação do supermercado para um dia de 12 horas (720 minutos), usando o seguinte algoritmo:

- 1) Escolha um inteiro aleatório entre 1 e 4 para determinar o minuto no qual o primeiro cliente chega.
- 2) No momento de chegada do primeiro cliente:

Determine a hora de atendimento do cliente (um inteiro aleatório de 1 a 4);

Comece a atender o cliente;

Programa a hora de chegada do próximo cliente (um inteiro aleatório de 1 a 4 somado à hora atual).

- 3) Para cada minuto do dia:

Se o próximo cliente chega, informe;

Coloque-o na fila;

Programa a hora de chegada do próximo cliente;

Se o atendimento ao último cliente foi completado:

Informe;

Retire da fila o próximo cliente a ser atendido;

Determine a hora de conclusão do atendimento ao cliente (um inteiro aleatório de 1 a 4 somado à hora atual).

Agora execute sua simulação para 720 minutos e responda às perguntas a seguir:

- a) Qual é o número máximo de clientes na fila, em qualquer momento?
- b) De quanto tempo foi a espera mais longa que um cliente teve de experimentar?
- c) O que acontece se o intervalo de chegada for mudado de 1 a 4 minutos para 1 a 3 minutos?

**12.16 Permissão para duplicatas em uma árvore binária.** Modifique o programa da Figura 12.19 para permitir que a árvore binária contenha valores duplicados.

**12.17 Árvore binária de busca de strings.** Escreva um programa baseado no programa da Figura 12.19 que aceite uma lista de texto, separe a sentença em palavras, insira as palavras em uma árvore binária de busca e imprima as travessias in order, pré-ordem e pós-ordem da árvore.

[Dica: leia a linha de texto para um array. Use strtok para separar o texto em palavras. Quando uma palavra for encontrada, crie um novo nó para a árvore, atribua o ponteiro retornado por strtok ao membro string do novo nó e insira o nó na árvore.]

**12.18 Eliminação de duplicatas.** Neste capítulo, vimos que a eliminação de duplicatas é simples quando se cria uma árvore binária de busca. Descreva como você realizaria a eliminação de duplicatas usando apenas um array subscrito. Compare o desempenho da eliminação de duplicatas baseada em array com o desempenho da eliminação de duplicatas baseada em árvore de busca binária.

**12.19 Profundidade de uma árvore binária.** Escreva uma função depth que receba uma árvore binária e determine seu número de níveis.

**12.20 Impressão recursiva de uma lista de trás para a frente.** Escreva uma função printListBackwards que envie recursivamente os itens de uma lista de trás



para a frente. Use sua função em um programa de teste que crie uma lista classificada de inteiros e imprima a lista em ordem reversa.

**12.21 Busca recursiva em uma lista.** Escreva uma função `searchList` que procure recursivamente um valor especificado em uma lista encadeada. A função deverá retornar um ponteiro para o valor que encontrou; caso contrário, `NULL` deverá ser retornado. Use sua função em um programa de teste que crie uma lista de inteiros. O programa deverá pedir ao usuário um valor a ser localizado na lista.

**12.22 Exclusão de árvore binária.** Neste exercício, discutimos a exclusão de itens das árvores de busca binária. O algoritmo de exclusão não é tão simples quanto o algoritmo de inserção. Há três possibilidades com as quais podemos nos deparar no caso de exclusão de um item: ele está contido em um nó folha (ou seja, não possui filhos), está contido em um nó que tem um filho ou está contido em um nó que tem dois filhos.

Se o item a ser excluído estiver contido em um nó folha, o nó é excluído e o ponteiro no nó pai é definido como `NULL`.

Se o item a ser excluído estiver contido em um nó com um filho, o ponteiro no nó pai é definido para apontar para o nó filho, e o nó contendo o item de dados é excluído. Isso faz com que o nó filho tome o lugar do excluído na árvore.

O último caso é o mais difícil. Quando um nó com dois filhos é excluído, outro nó precisa tomar seu lugar. Porém, o ponteiro no nó pai não pode simplesmente ser atribuído para apontar para um dos filhos do nó a ser excluído. Na maioria dos casos, a árvore binária de busca resultante não apoiaria a seguinte característica das árvores binárias de busca: *os valores em qualquer subárvore esquerda são menores que o valor no nó pai, e os valores em qualquer subárvore direita são maiores que o valor no nó pai.*

Qual nó é usado como um **nó substituto** para manter essa característica? O nó que contém o maior valor na árvore, menor que o valor no nó sendo excluído, ou o nó que contém o menor valor na árvore, maior que o valor no nó sendo excluído. Consideremos o nó de menor valor. Em uma árvore binária de busca, o maior valor, menor que o valor de um pai, está localizado na subárvore esquerda do nó pai, e tem garantias de estar contido no nó mais à direita da subárvore. Esse nó é localizado percorrendo-se a subárvore esquerda para a direita até que o ponteiro no filho da direita do nó atual seja `NULL`. Agora, estamos apontando para o nó substituto, que é um nó folha ou um nó com um filho à sua esquerda. Se

o nó substituto for um nó folha, as etapas para realizar a exclusão serão as seguintes:

- 1) Armazenar o ponteiro para o nó a ser excluído em uma variável de ponteiro temporária (esse ponteiro é usado para excluir a memória alocada dinamicamente).
- 2) Definir o ponteiro no pai do nó sendo excluído, para que aponte para o nó substituto.
- 3) Definir o ponteiro no pai do nó substituto como nulo.
- 4) Definir o ponteiro para a subárvore direita no nó substituto, para que aponte para a subárvore direita do nó a ser excluído.
- 5) Excluir o nó para o qual a variável de ponteiro temporário aponta.

As etapas de exclusão de um nó substituto com um filho esquerdo são semelhantes àsquelas de um nó substituto sem filhos, mas o algoritmo também deve mover o filho para a posição do nó substituto. Se o nó substituto for um nó com um filho esquerdo, as etapas da exclusão serão as seguintes:

- 1) Armazenar o ponteiro para o nó a ser excluído em uma variável de ponteiro temporária.
- 2) Definir o ponteiro no pai do nó a ser excluído para que aponte para o nó substituto.
- 3) Definir o ponteiro no pai do nó substituto para que aponte para o filho esquerdo do nó substituto.
- 4) Definir o ponteiro para a subárvore direita no nó substituto para que aponte para a subárvore direita do nó a ser excluído.
- 5) Excluir o nó para o qual a variável de ponteiro temporário aponta.

Escreva a função `deleteNode` que toma como argumentos um ponteiro para o nó raiz da árvore e o valor a ser excluído. A função deve localizar na árvore o nó que contém o valor a ser excluído, e usar os algoritmos discutidos aqui para excluir o nó. Se o valor não for encontrado na árvore, a função deverá imprimir uma mensagem que indique se o valor foi excluído ou não. Modifique o programa da Figura 12.19 para usar essa função. Depois de excluir um item, chame as funções de travessia `inOrder`, `preOrder` e `postOrder` para confirmar se a operação de exclusão foi realizada corretamente.

**12.23 Busca de árvore binária.** Escreva uma função `binaryTreeSearch` que tente localizar um valor especificado em uma árvore binária de busca. A função deve receber como argumentos um ponteiro para o nó raiz da árvore binária e uma chave de busca a ser localizada. Se o nó que contém a chave de busca for encontrado, a função deverá retornar um ponteiro para esse nó; caso contrário, a função deverá retornar um ponteiro `NULL`.

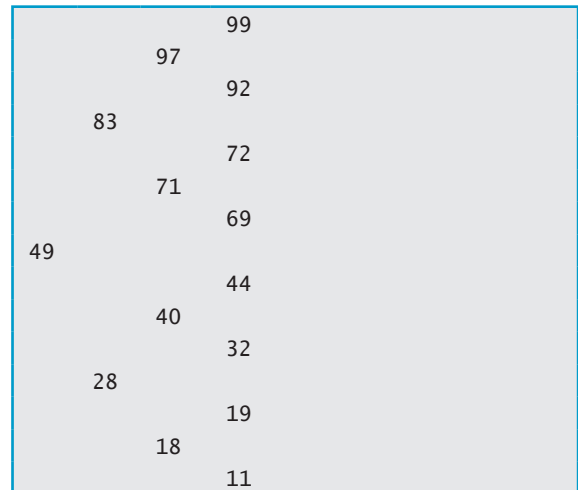
**12.24 Travessia de árvore binária em ordem de nível.**

O programa da Figura 12.19 ilustrou três métodos recursivos para atravessar uma árvore binária — na ordem, pré-ordem e pós-ordem. Esse exercício apresenta a **travessia em ordem de nível** de uma árvore binária, na qual os valores dos nós são impressos nível por nível, a começar pelo nível do nó da raiz. Em cada nível, os nós são impressos da esquerda para a direita. A travessia em ordem de nível não é um algoritmo recursivo. Ele usa a estrutura de dados de fila para controlar a impressão dos nós. O algoritmo é o seguinte:

- 1) Inserir o nó raiz na fila
- 2) Enquanto existirem nós na fila,
  - Obtenha o próximo nó na fila
  - Imprima o valor do nó
  - Se o ponteiro para o filho esquerdo do nó não for nulo
    - Insira o nó do filho esquerdo na fila
  - Se o ponteiro para o filho direito do nó não for nulo
    - Insira o nó do filho direito na fila

Escreva a função `levelOrder` para executar uma travessia em ordem de nível de uma árvore binária. Modifique o programa da Figura 12.19 para que seja possível usar essa função. Compare a saída dessa função com as saídas de outros algoritmos de travessia para se certificar de que ele funcionou corretamente. [Nota: você também terá que modificar e incorporar as funções de processamento de fila da Figura 12.13 nesse programa.]

**12.25 Impressão de árvores.** Escreva uma função recursiva `outputTree` para exibir uma árvore binária na tela. A função deve exibir a árvore linha por linha, com o topo da árvore à esquerda da tela e a parte inferior da árvore em direção ao lado direito da tela. Cada linha é exibida verticalmente. Por exemplo, a árvore binária mostrada na Figura 12.22 é exibida como:



Note que o nó folha mais à direita aparece no topo da saída da coluna mais à direita, e o nó raiz aparece na esquerda da saída. Cada coluna de saída começa cinco espaços à direita da coluna anterior. A função `outputTree` deve receber como argumentos um ponteiro para o nó raiz da árvore e um inteiro `totalSpaces` representando o número dos espaços que precedem o valor a ser exibido (essa variável deve começar em zero, de modo que o nó raiz seja exibido à esquerda da tela). Para exibir a árvore, a função usa uma travessia na ordem modificada — ela começa no nó mais à direita da árvore, e trabalha movendo-se para a esquerda. O algoritmo é o seguinte:

Enquanto o ponteiro para o nó atual não for nulo

Chame, recursivamente, `outputTree` com a subárvore direita do nó atual e `totalSpaces + 5`

Use uma estrutura `for` para contar de 1 até `totalSpaces`, e imprimir espaços

Exiba o valor no nó atual

Inicialize o ponteiro para o nó atual para que ele aponte para a subárvore esquerda do nó atual

Incremente `totalSpaces` em 5.

## Seção especial: a construção de seu próprio compilador

Nos exercícios 7.27 a 7.29, apresentamos a Simpletron Machine Language (SML), e você implementou um simulador de computador Simpletron para executar programas SML. Nos exercícios 12.26 a 12.30, construímos um compilador que converte programas escritos em uma linguagem de programação de alto nível para SML. Esta seção ‘amarra’ o processo de programação inteiro. Você escreverá programas nessa nova linguagem de alto

nível, usará o compilador que você mesmo construiu para compilá-los e os executará no simulador criado no Exercício 7.28. Você deverá se esforçar para implementar seu compilador de uma maneira orientada a objeto. [Nota: devido ao tamanho dos enunciados dos exercícios 12.26 a 12.30, preferimos incluí-los em um documento PDF em [www.deitel.com/books/cht6/](http://www.deitel.com/books/cht6/), em inglês.]